

Resumen Teórico Orientación a Objetos 1

La **programación Orientada a Objetos** es una forma de organizar problemas pensando en “cosas” del mundo real: usuarios, cuentas, autos, pedidos, personajes, etc.

Cada una de esas cosas tiene:

datos -> que lo sabe

comportamientos -> lo que puede hacer

Ej: Un Auto tiene:

datos -> color, velocidad, patente

comportamiento -> acelerar, frenar girar

La idea del paradigma OO es que el código quede más ordenado, entendible y fácil de modificar.

El paradigma de OO aporta:

1. Poder de abstracción.

Abstraer es quedarse con lo importante y ocultar los detalles innecesarios.

Cuando usas algo en la vida real, no necesitas entender cómo funciona por dentro.

Ej: Manejas un auto usando volante y pedales, no necesitas saber como explota el combustible dentro del motor.

2. Utilizando el dominio como guía.

El dominio es el problema real que estas resolviendo.

Ej: Un banco, una tienda online, un hospital, un videojuego.

OO intenta que el código se parezca al mundo real del problema.

Ej: Si tengo un sistema bancario, voy a tener algo como:

Cliente, cuenta, transferencia, tarjeta.

3. Juntando datos y comportamiento.

Esta es una de las bases más importantes de OO.

La idea es que el Objeto debe tener SUS datos y también las operaciones sobre esos datos.

4. Buena organización de nuestro programa.

OO ayuda a dividir el sistema en piezas más claras.

En vez de:

Un archivo gigante y miles de funciones mezcladas.

Tenés:

Clases, responsabilidades separadas, objetos especializados

5. Alta cohesión.

Cada clase tiene un objetivo claro. Cada clase debería responder ¿Para que existe? y

la respuesta debería ser ¿Por qué sirve? porque el código es más fácil de entender, más fácil de modificar, más fácil de reutilizar.

6. Bajo acoplamiento.

El acoplamiento es cuanto depende una parte del sistema de otra.

Mucho acoplamiento = problema. Si cambiar una clase rompe 20 mas el sistema está muy acoplado.

7. Localidad de cambios.

Cuando necesitas modificar algo deberías tocar pocas partes del sistema, si para hacer un cambio tenés que tocar muchas partes del sistema es un problema.

8. Extender el sistema agregando, sin tocar lo que ya funciona.

Esto es muy importante en sistemas grandes, la idea es agregar funcionalidades nuevas sin romper lo viejo.

Criterios y heurísticas de diseño y programación OO.

El objetivo de estas “reglas” es evaluar críticamente en detalle los diseños y programas orientados a objetos.

Malos Olores de diseño

- **Envidia de atributos:** Soy un objeto que pide cosas a otros objetos para hacer algo yo mismo (por ejemplo: un cálculo).

Para evitarlo: la tarea la debe hacer el objeto que tiene las cosas que se necesitan; delegárselo a él.

Ejemplo en Java:

Envidia de atributos:

```
class Persona {
    int edad;
}

class Calculador {
    boolean esMayor(Persona p) {
        return p.edad >= 18;
    }
}
```

Sin envidia de atributos:

```
class Persona {
    int edad;

    boolean esMayor() {
        return edad >= 18;
    }
}
```

Si un objeto vive preguntándole cosas a otros, probablemente hay envidia de atributos.

- **Clase Dios:** Una clase que resuelve todo y las demás están todas anémicas, una clase así no cumple el principio de una sola responsabilidad. Probablemente también haya envidia de atributos si es que otros objetos, al menos, tienen información.
Para evitarlo: ver que otros objetos podría hacer aparecer, que se puedan encargar de alguna de las responsabilidades de este. ver cuál de los objetos que este objeto conoce podría ser responsable de algo que ahora hace el.
- **Código duplicado:** Si hago Ctrl+C Ctrl+V (copiar y pegar) estoy metiendo la pata.
Para evitarlo: ¿No puedo generalizar ese comportamiento en una clase y heredarlo? ¿No puedo llevarlo a otro objeto y re-utilizarlo por composición? ¿No puedo extraerlo en un método en la misma clase y re-usarlo?
- **Clase larga:** Tengo una clase muy grande en comparación al resto.
Para evitarlo: ¿No será que esa clase puede delegar algo en otros objetos a los que conoce? ¿No será que esa clase modela más de una cosa? (Puedo pensarla como una composición de varios objetos).
- **Método largo:** Si un método tiene más de 10 renglones, es mala señal. Si debo incluir comentarios en medio de un método, es mala señal.
Para evitarlo: Identificar dentro del método largo, partes que podría considerar comportamientos individuales. Llevar cada parte a un nuevo método (con un buen nombre) y cuando necesite llevar a cabo uno de esos comportamientos, enviar mensajes a **this**.
- **Objetos que conocen el id de otro:** Nunca relacionar objetos por medio de claves o ids!!
Para evitarlo: Cuando un objeto se relaciona con otro, lo hace con una referencia. Nunca conoce su id (incluso aunque los objetos tengan id).
Ejemplo:

Mal:

```
class Pedido {
    int clienteId;
}
```

Bien:

```
class Pedido {
    Cliente cliente;
}
```

- **Eso debería ser un objeto (obsesión por los primitivos):** A veces modelamos como strings o números cosas que deberían ser Objetos. Cuando hacemos eso, el comportamiento que debería tener ese objeto termina estando en un lugar que no corresponde.

Para evitarlo: Pensar si eso que estoy modelando con un string o número (primitivo) no debería ser modelado con una clase específica.

- **Switch statements:** Debería sentir mal olor cuando veo que se usa un **if** (o algo que parece un **case** o un **Switch** o **ifs anidados**) para determinar de qué forma se resuelve algo. Esto es más evidente si la variable que uso en el **if** tiene un nombre que sea a "tipo". Algo como "if (tipo = esto) entonces lo hago así, pero si (tipo = aquello) entonces lo hago así" tiene muy feo olor. El caso más extremo es preguntar por la clase del objeto (si es de esta clase lo hago así, si no lo hago así).

Para evitarlo: ¡Aplico adecuadamente polimorfismo!

Ejemplo:

```
if (tipo.equals("PERRO")) {  
    hacerGuau();  
}  
else if (tipo.equals("GATO")) {  
    hacerMiau();  
}
```

Problema:

Cada vez que agrego un animal tengo que modificar el código, para solucionarlo hago una clase animal abstracta con un método abstracto hacer sonido y después clases correspondientes con los animales con sus respectivos comportamientos.

- **Variables de instancia que en realidad deberían ser temporales:** Si una variable de instancia deja de tener sentido en algún momento de la vida del objeto, entonces es probable que sea temporal o que sea responsabilidad de otro.

Para evitarlo: pensar si esa variable es realmente un atributo del objeto, que lo acompaña siempre o es algo que necesito temporalmente dentro de un método.

- **Romper encapsulamiento:** Romper el encapsulamiento de un objeto es muy malo, Nos hace perder la gran mayoría de las ventajas de la OO. No solo es preocupante acceder a variables de instancia de otros objetos, además podemos romper el encapsulamiento de manera más sutil. Por ejemplo, si automáticamente agregamos setters y getters para todas las variables de instancia de nuestros objetos, estamos invitando a otros a que las modifiquen cuanto quieran (como si no existiera un ocultamiento de información) al modificar objetos o colecciones que son de otros. Si modificamos una colección que no es nuestra (es de otro objeto) también atentamos contra el encapsulamiento.

Para evitarlo: solo agregar getters y setters cuando es necesario; nunca modificar una colección que no es nuestra, delegar las tareas a los que tienen la información que se necesita.

- **Clase de datos o clase anémica:** una clase que parece un registro de datos debería dar mala espina. A veces los enunciados son simplificaciones que hacen que algunas clases terminen siendo así, pero por lo general sospecho cuando una clase solo tiene datos y no tiene comportamiento.
Para evitarlo: asegurarse que no hay comportamiento en el sistema que debería estar haciendo esa clase y lo hace otro objeto (el cual seguramente muestre envidia de atributos).

Ejemplo:

Mal

```
class Cuenta {
    double saldo;
}

class Banco {
    void retirar(Cuenta c, double monto) {
        c.saldo -= monto;
    }
}
```

Bien

```
class Cuenta {
    double saldo;

    void retirar(double monto) {
        saldo -= monto;
    }
}
```

- **No es-un:** Una relación de herencia (clase B hereda de clase A) siempre debe respetar el principio **es-un**. Si me pregunto “¿un B es un A?”, la respuesta debe ser SI, si la respuesta es NO, eso tiene mal olor.

Para evitarlo: Siempre preguntarme “¿es un?” cuando defino una subclase. Si la respuesta es no, pensar un poco más. A veces el problema es que elegí mal los nombres de las clases. A veces es señal de que tanto A como B son subclase de otra clase que todavía no apareció. A veces es señal de que la relación de subclasificación no es la correcta para ese caso, y debo pensar otras alternativas (como composición).

Ejemplo:

Auto hereda de Motor -> Auto es un motor? No

Solución:

Auto tiene un Motor -> Composición.

- **No quiero mi herencia:** cuando encontramos un método que redefina a uno heredado, pero hace algo totalmente diferente, debemos desconfiar. Si no sirve el comportamiento heredado tal vez no se cumpla el principio “es-un”. El caso extremo es redefinir un método heredado, para indicar un error o no hacer nada.

Para evitarlo: pensar si no puedo reorganizar la jerarquía de clases para que ninguna herede comportamiento que no quiere.

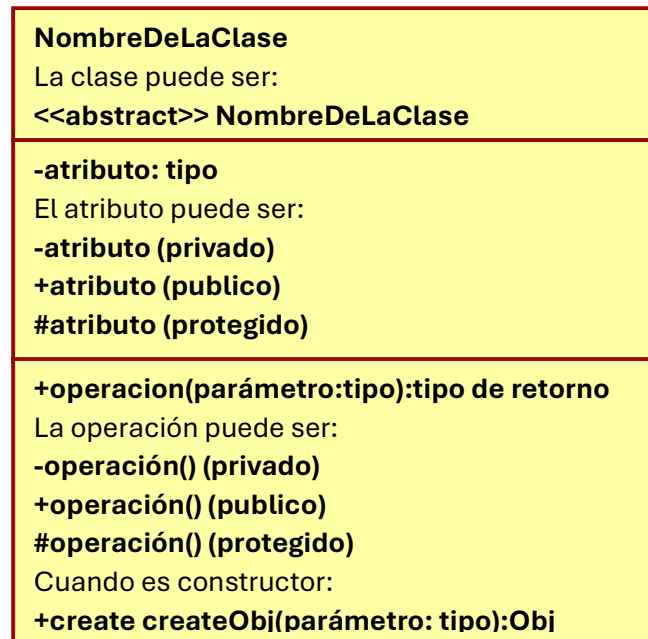
- **Reinventando la rueda:** un principio fundamental de la POO es que las cosas se escriben una sola vez y donde corresponde. De esa manera, mis módulos (objetos/métodos) son más fáciles de mantener y reutilizar. Tiene mal olor cuando defino comportamiento que sospecha que ya está programado en algún lado (hay algún objeto que ya sabe hacer eso). Un ejemplo en Java sería implementar en base a "for (int it= 0;...; i++)" cosas que las colecciones, los iteradores y los streams ya saben hacer.
Para evitarlo: investigo y aprendo las clases y protocolos que ofrecen las librerías de objetos a mi disposición. Intento siempre utilizar comportamiento que ya fue definido. Presto especial atención cuando utilizo colecciones, fechas, etc.

Estilo de Programación:

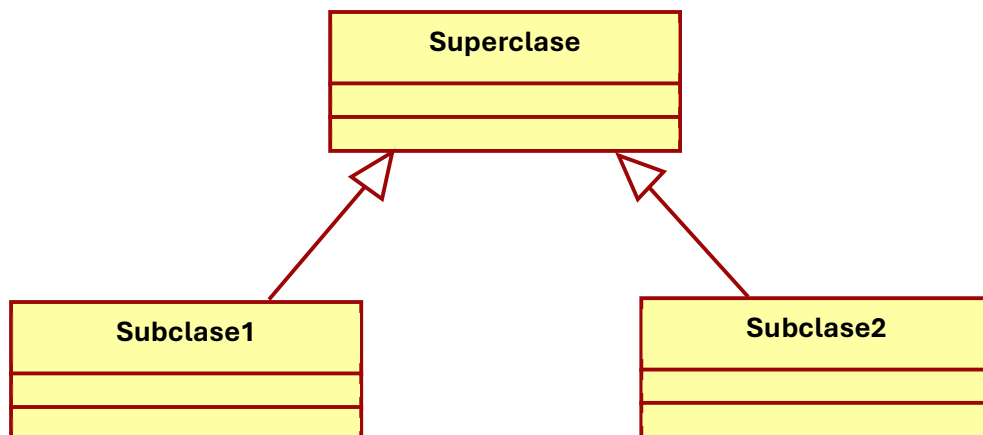
- **Ofrecer constructores:** Simplifica la tarea de quien crea los objetos. Garantizan una buena inicialización.
- **Nombre de mensaje que revela la intención:** Que el nombre del mensaje comunique lo que quiere hacer, no como.
- **Delegación a this:** Permite descomponer un método en partes que el mismo objeto resuelve. Cada método hace una cosa. Su nombre indica lo que hace. Quedan todos cortos. Permite que la subclase redefina/extienda solo un paso.
- **Métodos cortos:** Siempre prefiere tener métodos cortos. Para lograrlo utiliza delegación a this. Siempre nombres fáciles de leer que revelen la intención.
- **Cada cosa se hace una sola vez:** Para ello es importante aprender el protocolo de colecciones y otros objetos frecuentemente utilizados.
- **Los nombres de las variables deben indicar su rol:** Elige los nombres de las variables para que quede claro que rol cumplen en el método/clase. Los nombres de las variables siempre comienzan con minúscula.
- **Pensar bien los nombres de las clases:** Esos siempre inician con mayúscula y singular no temas a los nombres de clase largos, con varias palabras y estilo CamelCase

Diagrama de clases UML

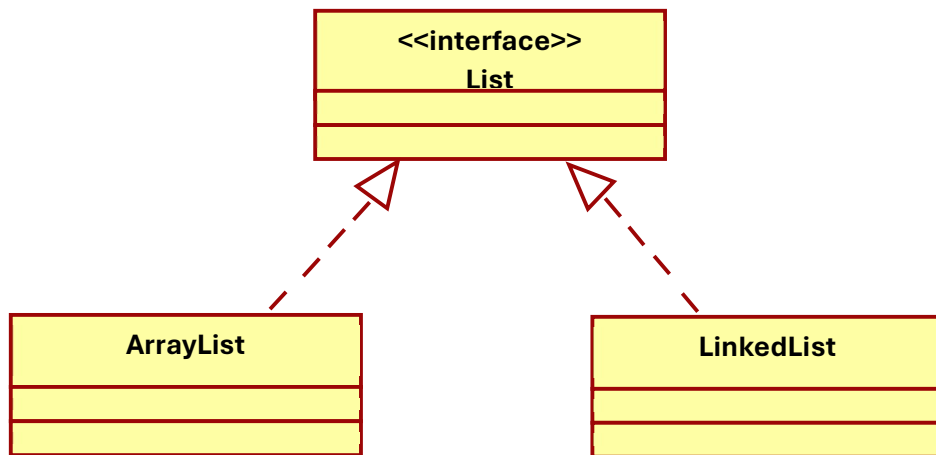
Como diagramar una clase:



Herencia:



Interfaces:



Asociaciones entre clases:

***Para las asociaciones es un buen tip preguntarse ¿Quién necesita conocer a quien?**

Asociacion Bidireccional (ambas clases se conocen)



Asociacion Unidireccional (con flecha) (una de las clases conoce a la otra)



En este caso Invitación necesita Archivo, pero Archivo no necesita a Invitación.
La Invitación tiene una referencia al Archivo.

Cardinalidades:

Las cardinalidades indican cuantos objetos de una clase se relacionan con otra:

1 -> **exactamente uno**

0..1 -> **opcional (puede no haber)**

* -> **muchos**

1..* -> **al menos uno**



Asociacion

En UML, una asociacion representa una relacion entre clases (objetos que “se conocen” o interactuan).

Navegabilidad:

Indica desde que clase se puede acceder a la otra, Se representa con una flecha en la asociacion.

Ejemplo: **Cliente** -----> **Pedido**

Significa que un Cliente conoce sus Pedido, pero no necesariamente al reves.

Unidireccional: ----->

Bidireccional: ----- (sin flecha)

Multiplicidad:

Indica cuantos objetos de una clase pueden relacionarse con objetos de la otra:

1 -> exactamente uno

0..1 -> cero o uno

* -> muchos

1..* -> uno o muchos

Ejemplo: **Cliente** 1-----* **Pedido**

Significa que in Cliente puede tener muchos pedidos, pero cada pedido pertenezca a un solo cliente

Nombre de rol

Es el nombre que toma una clase dentro de la relacion, ayuda a entender el papel que cumple cada extremo.

Ejemplo: **Empresa** -----empleado----- **Persona**

Significa que la clase Persona cumple el rol de empleado dentro de la empresa.

Tipos de asociacion

Asociacion simple

Relacion normal entre clases.

Profesor ----- **Curso**

No implica dependencia fuerte de vida

Agregacion ◇

Relacion “todo-parte” debil. Se representa con un rombo blanco.

Equipo ◇----- **Jugador**

El Equipo tiene Jugador, pero los jugadores pueden existir sin el Equipo.

La parte puede vivir sin el todo

Composicion ◆

Relacion “todo-parte” fuerte. Se representa con un rombo negro.

Casa ◆----- **Habitacion**

La Habitacion depende de la Casa, la casa desaparece, desaparecen las habitaciones.

La parte no tiene sentido sin el todo.

Herencia

Mecanismo que permite a una clase “heredar” estructura y comportamiento de otra clase.

Es una estrategia de reuso de código, es una estrategia de reuso de conceptos/definiciones. En Java solo hay herencia simple, es una característica transitiva.

```
public class CuentaCorriente extends CuentaBancaria {
```



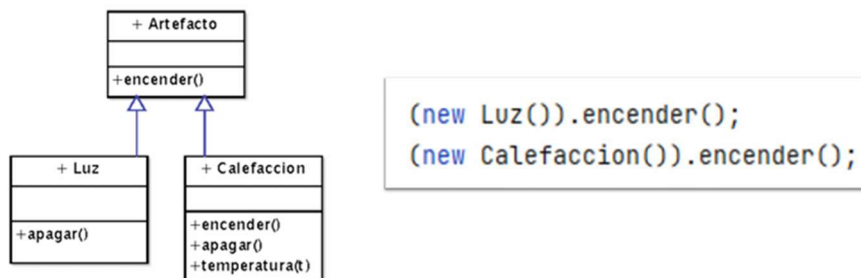
Metodo Lookup

Cuando un objeto recibe un mensaje, se busca en su clase un método cuya firma se corresponda con el mensaje.

- En un lenguaje dinámico, podría no encontrarlo (error en tiempo de ejecución)
- En un lenguaje con tipado estático sabemos que lo entenderá (aunque no sabemos lo que hará)

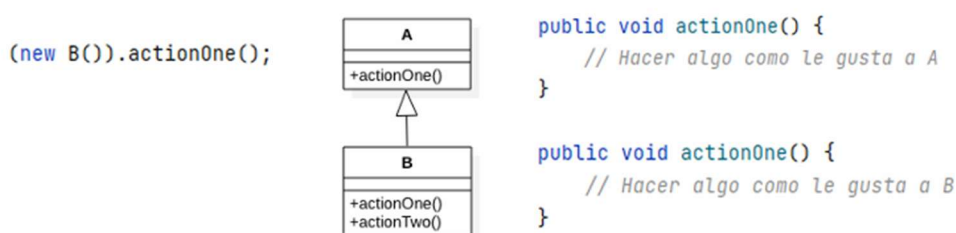
Metodo Lookup con herencia

Cuando un objeto recibe un mensaje, se busca en su clase un método cuya firma se corresponda con el mensaje. Si no lo encuentra, sigue buscando en la superclase de su clase, y en la superclase de esta y así sucesivamente.



Sobreescribir metodos. (overriding)

La búsqueda en la cadena de superclases termina tan pronto encuentre un método cuya firma coincide con la que busco. Si heredaba un método con la misma firma, el mismo queda “oculto” (redefinir / override) • No es algo que ocurra con frecuencia (puede dar mal olor).

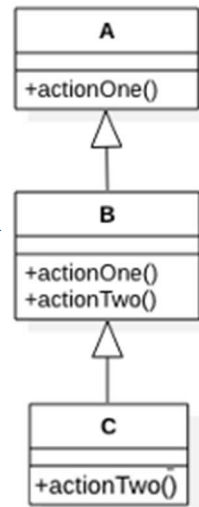


Super y metodo lookup

Cuando el **super** recibe un mensaje, la búsqueda de metodos comienza en la clase **inmediata superior** a aquella donde esta definido el metodo que envia el mensaje (sin importar la clase del receptor).

```
(new A()).actionOne();
(new B()).actionOne();
(new C()).actionOne();

public void actionOne() {
    this.actionTwo();
    super.actionOne();
    this.actionTwo();
}
```



Super() en los constructores

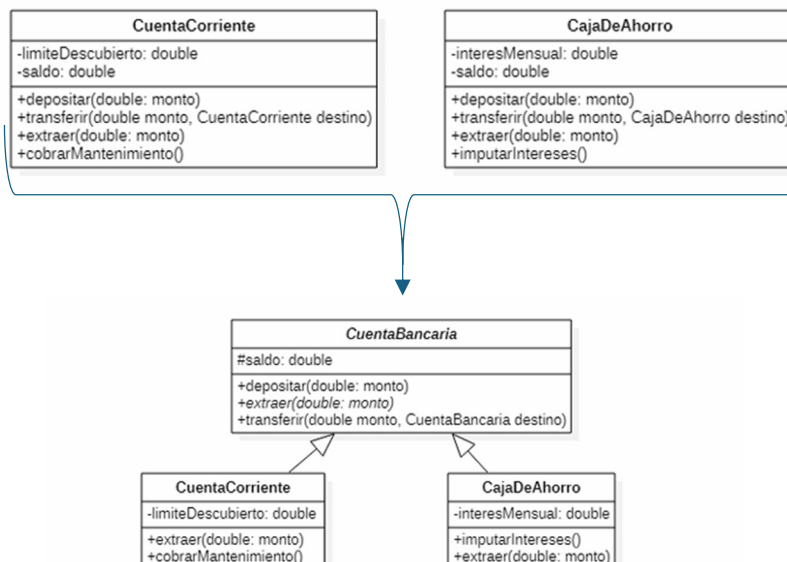
Los constructores en Java son subrutinas que se ejecutan en la creacion de objetos – no se heredan. Si quiero reutilizar comportamiento de otro constructor debo invocarlo explicitamente – usando **super()** al principio.

```
public CuentaBancaria(Persona titular) {
    this.titular = titular;
}

public CuentaCorriente(Persona titular, double saldoInicial, double limiteDescubierto) {
    super(titular);
    saldo = saldoInicial;
    this.limiteDescubierto = limiteDescubierto;
}
```

Generalizar

Introducir una superclase que abstraiga aspectos comunes a otras – suele resultar en una clase abstracta.



Clases abstractas e interfaces

- Una clase abstracta “es una clase”
 - Puedo usarla como tipo
 - Puede o no tener métodos abstractos
 - Ofrece implementación a algunos métodos
 - Sus métodos concretos pueden depender de sus métodos abstractos (p.e., si se define antes de sus subclasses)
- Una interfaz define un tipo
 - Sirve como contrato
 - Puede extender a otras
 - Se pueden implementar muchas interfaces pero solo se puede heredar de una clase

Modificadores de visibilidad con herencia

- Si declaro una variable de instancia como private en una clase A:
 - Las instancias de las subclasses de A tendrán esa variable de instancia.
 - En los métodos de las subclasses de A, no puedo hacer referencia ella.
- Si declaro un método “m()” como private en una clase A:
 - Las instancias de sus subclasses entenderán el mensaje m()
 - En los métodos de las subclasses de A no puedo enviar m() a “this”
 - Si declaro variables y métodos como protected, puedo verlos en las subclasses

Contratos

Son una forma de describir el comportamiento en un sistema en forma detallada. Describen **pre** (antes) y **post** (después) condiciones.

Secciones en un contrato

Operación: se detalla el nombre de la operación y los parámetros

- **Pre-condiciones:** Suposiciones relevantes sobre el estado del sistema o de los objetos del Modelo del Dominio, antes de la ejecución de la operación.
 - No se validan dentro de la operación, sino que se asumen válidas.
 - Son suposiciones no triviales.Ej: La cantidad de días de contratación de un servicio prolongado es mayor a 1.
- **Post-condiciones:** El estado del sistema o de los objetos del Modelo del Dominio, después de que se complete la ejecución de la operación.
 - Describen cambios en los objetos del modelo de dominio.

- Modificación del valor de atributos.
 - Creación o eliminación de instancias.
 - Creación o ruptura de asociaciones.
 - Son declarativas
- Ej: El cliente posee una nueva contratación.

this (un objeto que habla solo)

this (o en algunos lenguajes self) es una “pseudo-variable” no puedo asignarle valor
Toma valor automáticamente cuando un objeto comienza a ejecutar un método

this hace referencia al objeto que ejecuta el método (al receptor del mensaje que resultó en la ejecución del método)

Se utiliza para:

- Descomponer métodos largos (refinamiento top-down)
- Reutilizar comportamiento repetido en varios métodos
- Aprovechar comportamiento heredado
- Pasar una referencia para que otros puedan enviarnos mensajes

Igualdad == y método equals()

```
// implementación "aproximada" de equals en la clase Color
public boolean equals(Color otro) {
    return this.getRed() == otro.getRed() &&
           this.getGreen() == otro.getGreen() &&
           this.getBlue() == otro.getBlue();
}
```

```
Color blanco = new Color(255, 255, 255);
Color otroBlanco = new Color(255, 255, 255);
Color unColor = blanco;
```

```
System.out.println(blanco == blanco); // true
System.out.println(blanco == unColor); // true
System.out.println(blanco == otroBlanco); // false
System.out.println(blanco.equals(unColor)); // true
System.out.println(blanco.equals(otroBlanco)); // true
System.out.println(blanco.equals(new Color(0,0,0))); // false
```

Tipos en lenguajes OO

-Tipo: Conjunto de firmas de operaciones/métodos (nombre, orden y tipos de los argumentos)

Algunos lenguajes diferencian entre tipos primitivos y tipos de referencias (objetos)

Puedo utilizar clases, para dar tipo a las variables Asignar un objeto a una variable, no afecta al objeto (no cambia su clase)

La clase de un objeto se establece cuando se crea, y no cambia más Pero ... las clases no son la única forma de definir tipos.

Interfaces

Una clase define un tipo, y también implementa los métodos correspondientes.

Una variable tipada con una clase solo "acepta" instancias de esa clase (*).

Una interfaz nos permite declarar tipos sin tener que ofrecer implementación. (desacopla tipo e implementación).

Puedo utilizar Interfaces como tipos de variables.

Las clases deben declarar explícitamente que interfaces implementan.

Una clase puede implementar varias interfaces.

Ejemplo:

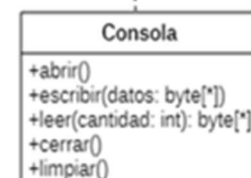
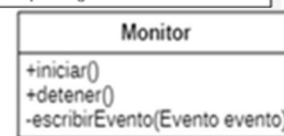
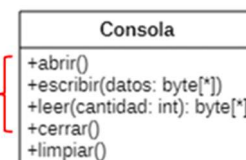
Problema:

```
public class Monitor {  
    private ??? destino;  
  
    public Monitor( ??? destino) {  
        this.destino = destino;  
    }  
  
    private void escribirEvento(Evento evento) {  
        destino.escribir(evento.asBytes());  
    }  
}
```

¿Nos interesa decir que de clase es "destino" o que mensajes entiende?

Solución:

```
public class Monitor {  
    private Canal destino;  
  
    public Monitor(Canal destino) {  
        this.destino = destino;  
    }  
  
    private void escribirEvento(Evento evento) {  
        destino.escribir(evento.asBytes());  
    }  
}  
  
public interface Canal {  
    public void abrir();  
    public void escribir(byte[] datos);  
    public byte[] leer(int cantidad);  
    public void cerrar();  
}  
  
public class Consola implements Canal {  
  
public class Archivo implements Canal {
```



Polimorfismo

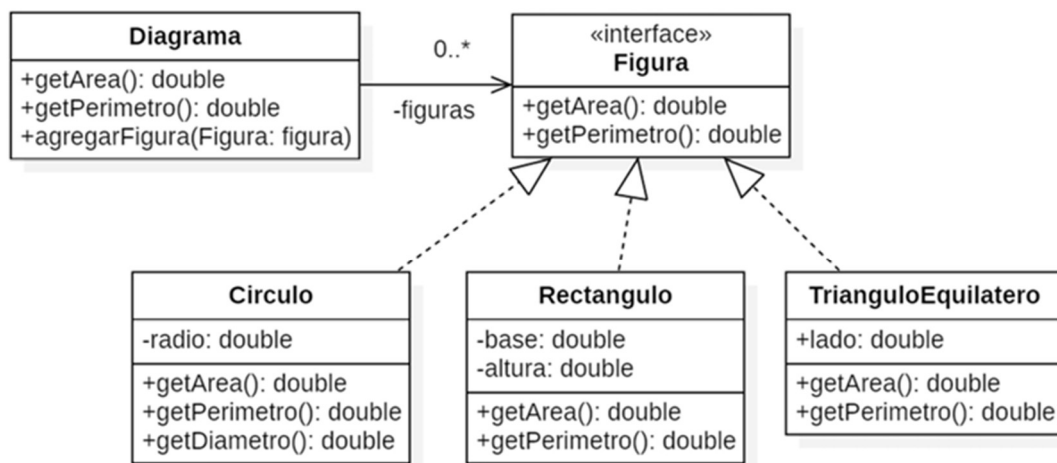
Objetos de distintas clases son polimórficos con respecto a un mensaje, si todos lo entienden, aun cuando cada uno lo implemente de un modo diferente.

Polimorfismo implica:

- Un mismo mensaje se puede enviar a objetos de distinta clase.
- Objetos de distinta clase “podrían” ejecutar métodos diferentes en respuesta a un mismo mensaje.

Cuando dos clases Java implementan una interfaz, se vuelven polimórficas respecto a los métodos de la interfaz.

Ejemplo polimórfico:



El polimorfismo permite:

- Permite repartir mejor las responsabilidades (delegar).
- Desacopla objeto y mejora la cohesión (cada cual hace lo suyo).
- Concentra cambios (reduce el impacto de los cambios).
- Permite extender sin modificar (agregando nuevos objetos).
- Lleva a código más genérico y objetos reusables.
- Nos permite programar por protocolo, no por implementación.

Testing

Test de unidad

Test que asegura que la unidad mínima de nuestro programa funciona correctamente, y aislada de otras unidades.

- En nuestro caso, la unidad de teste es el método.

Testear un método es confirmar que el mismo acepta el rango esperado de entradas, y que retorna el valor esperado en cada caso.

Tengo en cuenta:

- Parametros.
- Estado del objeto antes de ejecutar el método.
- Objeto que retorna el método.
- Estado del objeto al conducir la ejecución del método.

Test automatizados

Se utiliza en software para guiar la ejecución de los test y controlar los resultados.

Requiere que diseñemos, programemos y mantengamos programas "test".

- En nuestro caso esos programas serán objetos.

Suele basarse en herramientas que resuelven gran parte del trabajo.

Una vez escritos, los pedo reproducir a costo mínimo, cuando quiera.

Los test son "parte del software" (y un indicador de su calidad).

jUnit

jUnit es una herramienta para simplificar la creación de test de unidad y automatizar su ejecución y reporte.

Ayuda a escribir/programar test utiles.

Cada test se ejecuta independientemente de otros(aislados).

jUnit detecta, recolecta y reporta errores y problemas.

xUnit es su nombre genérico. (lo mismo para otros lenguajes).

Anotmía de un test suite jUnit

- Una clase de test por cada clase a testear.

- Un método que prepara lo que necesitan los test (el fixture).

- Uno o varios métodos de test por cada método a testear.

- Un método que limpia lo que se preparó si es necesario.

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class PersonaTest {

    Persona james;

    @BeforeEach
    void setUp() throws Exception {
        james = new Persona();
        james.setApellido("Glosing");
        james.setNombre("James");
    }

    @Test
    public void testNombreCompleto() {
        assertEquals("Glosing, James",
            james.getNombreCompleto());
    }
}
```


Ejemplo: El robot

- El robot avanza y retrocede de a un lugar
- En cada movimiento consume una unidad de energía.

Robot
-position: int -energy: int
+getPosition(): int +getEnergy(): int +goForward() +goBackwards() -consumeEnergy()

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class RobotTest {

    private Robot robot;

    @BeforeEach
    void setUp() {
        robot = new Robot(0,100);
    }

    @Test
    void testGoForward() {
        robot.goForward();
        assertEquals(1, robot.getPosition());
    }

    @Test
    void testGoBackwards() {
        robot.goBackwards();
        assertEquals(-1, robot.getPosition());
    }
}
```

Importamos las partes de JUnit que necesitamos

Definición y preparación del "fixture"

Ejercitar los objetos
Verificar resultados

Ejercitar los objetos
Verificar resultados

Tests

Algunas Variantes del **assert**

```

@Test
void assertExamples() {
    assertEquals(5, "Hello".length());
    assertNotEquals("Hello", "Bye");
    assertNotNull(myList);
    assertSame(myList, someList);
    assertTrue(myList.isEmpty());
    assertFalse(someList.isEmpty());
    assertThrows(IndexOutOfBoundsException.class, () -> {
        myList.remove("Hello");
    });
}

```

Test de particiones equivalentes

Partición de equivalencia: conjunto de casos que prueban lo mismo o revelan el mismo bug.

-Asumo que si un ejemplo de una partición pasa el test, los otros también lo harán. Elijo uno.

Si se trata de casos en un conjunto, tomo un caso que pertenezca al conjunto y uno que no.

-Ej: debe tener entrada -> Casos una persona con entrada, una sin.

Si se trata de valores en un rango, tomo un caso dentro y uno por fuera en cada lado del rango.

-Ej: la temperatura debe estar entre 0 y 100 -> casos -50, 50, 150.

Estos casos pueden mejorarse.

Preguntas que nos debemos hacer a la hora de hacer las participaciones equivalentes.

- ¿Qué tests deberíamos escribir? (Qué clases y qué métodos)
- ¿Qué particiones identificamos?
- ¿Cuáles serían buenos casos de test para cada una?
- ¿Podemos pensar otros casos que prueben algo diferente?

Ej: El robot minimalista

- El robot avanza y retrocede de a un lugar sin importarle nada
- Identificamos dos métodos a testear.
- Identificamos una partición.
- Cualquier robot da lo mismo.

```

class MinimalRobotTest {

    private MinimalRobot robot;

    @BeforeEach
    void setUp() {
        robot = new MinimalRobot();
    }

    @Test
    void goForward() {
        robot.goForward();
        assertEquals(1, robot.getPosition());
    }

    @Test
    void goBackwards() {
        robot.goBackwards();
        assertEquals(-1, robot.getPosition());
    }
}

```

MinimalRobot
-position: int
+getPosition(): int +goForward() +goBackwards()

Ej: El robot apagable

- Se puede encender y apagar (con un mismo mensaje).
- Si está apagado, no hace nada al pedirle que se mueva.
- Identificamos dos métodos a testear.
- Identificamos dos particiones (encendido, en cualquier lugar y apagado, en cualquier lugar).
- Cuatro casos en total (podemos tener varios casos en un método de test, también podemos separar casos en métodos).

ToggleableRobot
-position: int -isOn: boolean
+getPosition(): int +toggle() +goForward() +goBackwards()

```

class ToggleableRobotTest {
    private ToggleableRobot onRobot, offRobot;

    @BeforeEach
    void setUp() {
        onRobot = new ToggleableRobot();
        onRobot.toggle();
        offRobot = new ToggleableRobot();
    }

    @Test
    void testGoForward() {
        //Partition of robots that are turned on
        onRobot.goForward();
        assertEquals(1, onRobot.getPosition());
        //Partition of robots that are turned off
        offRobot.goForward();
        assertEquals(0, offRobot.getPosition());
    }

    @Test
    void testGoBackwards_onRobots() {
        onRobot.goBackwards();
        assertEquals(-1, onRobot.getPosition());
    }
}

```

Varios casos en un método de test

Separar casos en métodos

Test con valores de borde

Los errores ocurren con frecuencia en los límites y ahí es donde los vamos a buscar. Intentamos identificar bordes en nuestras particiones de equivalencia y elegimos valores.

Buscar los bordes en estados/parámetros: velocidad, cantidad, posición, tamaño, duración, edad, etc.

También se puede buscar relaciones entre ellas (diferencia entre saldo y monto a extraer).

Buscar valores como: primero/último, máximo/mínimo, arriba/abajo, principio/fin, vacío/lleño, antes/después, junto a, alejado de, etc.

Ej: El robot positivista

- El robot avanza y retrocede de a un lugar, pero solo en positivos (desde 0 a Integer.MAX_VALUE).
- Identificamos dos métodos a testear.
- Las particiones/bordes pueden ser diferentes para distintos métodos.

PositivistRobot
-position: int
+getPosition(): int +goForward() +goBackwards()

```
class PositivistRobotTest {
    PositivistRobot robotAtZero, robotAtMax, robotInBetween;
    @BeforeEach
    void setUp() {
        robotAtZero = new PositivistRobot();
        robotAtMax = new PositivistRobot(Integer.MAX_VALUE);
        robotInBetween = new PositivistRobot(100);
    }

    @Test
    void testGoBackwards_inBetween() {
        robotInBetween.goBackwards();
        assertEquals(99, robotInBetween.getPosition());
    }

    @Test
    void testGoBackwards_atZero() {
        robotAtZero.goBackwards();
        assertEquals(0, robotAtZero.getPosition());
    }

    @Test
    void testGoForward_inBetween() {...}
    @Test
    void testGoForward_atMax() {...}
}
```

Ej: El saltarín y hambriento

- Tiene energía (que puede ser 0) y tiene hambre (mínimo 1).
- Cada lugar que se mueve indica tanta energía como su hambre indica.
- Si no tiene energía suficiente se queda en el lugar.
- En lugar de avanzar y retroceder de a uno, salta.

Se pide: “identifique, especifique y justifique los casos de test”:

- Identifico tres métodos a testear: charge y los dos jump
- ¿Qué particiones/bordes encontramos para cada método? -> Pienso en combinaciones de carga, hambre y cantidad de lugares
- testChargue()
 - position y hunger son irrelevantes.
 - Cualquier combinación energía/amount sirve. Elijo: energy=0 ; amount=1;
- testJump()
 - Considero la relación energy, hunger y places.
 - Partición sin suficiente energía: Elijo uno de los casos mínimos: energy=1 ; hunger=1 ; places=1;
 - Partición con suficiente energía: Elijo uno de los casos mínimos: energy=1 ; hunger=1 ; places=1;

JumpingHungryRobot
-position: int -hunger: int -energy: int
+getPosition(): int +jumpForward(places: int) +jumpBackwards(places: int) +charge(amount: int): int +setHunger(hunger: int)

Particiones comunes:

Colecciones

- vacía
- un elemento
- varios elementos

Búsquedas

- encontrado
- no encontrado
- varios encontrados

Cálculos

- resultado 0
- resultado positivo

Strings

- vacío
- coincide
- no coincide

Uso de Colecciones en Java, Lambda expresiones y Streams

Colecciones

Java provee un framework de colecciones muy rico conformado por un conjunto de interfaces y clases que las implementan.

ArrayList

Una instancia de la clase `java.util.ArrayList` constituye una colección lineal ordenada crece dinámicamente y puede contener elementos duplicados. Ofrece acceso posicional en tiempo constante. Esta indexada comenzando desde cero: es decir que el primer elemento de la lista se encuentra en esa posición.

Permite agregar y eliminar elementos. Cada vez que se agrega un elemento, la lista lo ubica en la última posición; no obstante, esto, pueden agregarse elementos en otras posiciones, lo cual genera el desplazamiento de una posición en sentido creciente de todos los elementos que se encuentren ubicados a partir del lugar en el que se agrega el nuevo. Pueden eliminarse elementos de cualquier posición de la lista. Cuando esto ocurre, la lista genera un desplazamiento en sentido contrario al anteriormente descrito con el fin de no dejar espacios vacíos (por supuesto que este desplazamiento no ocurre cuando se elimina el último elemento de la lista).

Definición y creación:

```
List alumnos = new ArrayList();
```

Mensajes mas utilizados de ArrayList:

- `add(<T> object)`
Agrega el objeto "object", de tipo "T", a la última posición de la lista.
- `add(int index, <T> object)`
Agrega el objeto "object", de tipo "T", en la posición indexada con index (recordar que la primera posición es la cero). Si index no denota la última posición sino una intermedia, se realiza un corrimiento de los elementos existentes para hacer lugar al nuevo elemento.

- **get(int index)**
Devuelve el elemento que se encuentra en la posición indicada por index. Esto NO elimina el elemento de la lista.
- **size()**
Devuelve la cantidad de elementos que contiene la lista. Este número siempre superará en uno al índice de la posición del último elemento (puesto que el índice de las listas comienza en cero).
- **contains(<T> object)**
Devuelve true si la lista contiene el elemento "object", y false en caso contrario.
- **remove(int index)**
Elimina el elemento contenido en la posición indicada por index. Esto genera un corrimiento de los elementos ubicados en las posiciones superiores, de forma de no dejar lugar vacío.
- **clear()**
Elimina todos los elementos de la lista, dejándola vacía.
- **isEmpty()** Devuelve true si la lista está vacía, y false en caso contrario.

Expresiones Lambda

A partir de Java 8, se incorporó el soporte a expresiones lambda. Las expresiones lambda son funciones anónimas que no pertenecen a ninguna clase y son utilizadas porque necesitamos utilizar una funcionalidad una única vez. Normalmente, creamos expresiones lambda con el mero propósito de enviarla como parámetro a una función de alto orden (se entiende como función de alto orden a una función que recibe como parámetro a otra función).

El potencial de las expresiones lambda es enorme, y se utiliza en distintos escenarios a lo largo de una aplicación.

Sintaxis:

(parámetros, separados, por, coma) -> {cuerpo lambda}

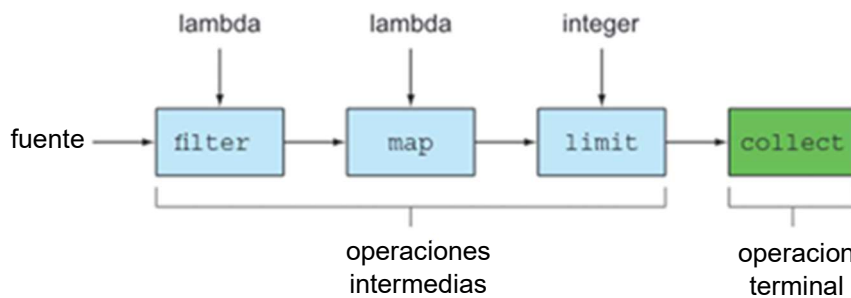
1. El operador lambda (->) separa la declaración de parámetros de la declaración del cuerpo de la función.
2. Parámetros:
 - Cuando se tiene un solo parámetro no es necesario utilizar los paréntesis.
 - Cuando no se tienen parámetros, o cuando se tienen dos o más, es necesario utilizar paréntesis. Si hay más de un parámetro, es necesario separarlos con coma.
3. Cuerpo de lambda:
 - Si el cuerpo de la expresión lambda tiene una única línea no es necesario

utilizar las llaves y no necesitan especificar la cláusula return en el caso que retorne algún valor.

- Si el cuerpo de la expresión lambda tiene más de una línea es necesario utilizar las llaves y es necesario incluir la cláusula return en el caso que retorne algún valor.

API Streams

Streams es una API de Java que nos va a facilitar el manejo de colecciones. Esto nos permite hacer operaciones como buscar, filtrar, sumar elementos de una colección de una forma sencilla. La API Streams define una amplia variedad de operaciones con la intención de permitir su encadenamiento de la siguiente forma:



En la figura anterior puede verse un ejemplo de su uso. A partir de la fuente, en nuestro caso será una colección, se aplicarán una serie de operaciones para obtener el resultado deseado. Vamos a utilizar streams junto con las expresiones lambda para operar con las colecciones de objetos.

Las operaciones definidas en la API se pueden clasificar en dos categorías: operaciones intermedias y operaciones terminales. Las operaciones intermedias retornan un stream para permitir el encadenamiento con otras operaciones. Las operaciones terminales permiten retornar un tipo diferente a Stream, como una Lista, un Integer o void.

Nota: si no se quiere retornar un stream, siempre se debe utilizar una operación terminal.

Solución tradicional vs solución utilizando streams y lambdas

Por ejemplo, si quisiéramos obtener los alumnos inscriptos a partir del 2020.

Solución tradicional:

```
List<Alumno> alumnosInscriptosDespues2020 = new ArrayList<Alumno>();
for (Alumno alumno: alumnos){
    if (alumno.getAñoIngreso() > 2020){
        alumnosInscriptosDespues2020.add(alumno);
    }
}
```

Solución con Streams y expresiones lambda:

```
List<Alumno> alumnosInscriptosDespues2020 = alumnos.stream()
    .filter(alumno -> alumno.getAñoIngreso() > 2020)
    .collect(Collectors.toList());
```

Operaciones intermedias

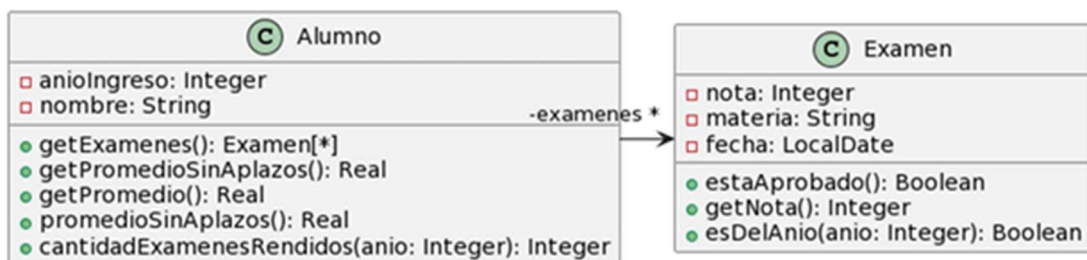
filter
map
limit
sorted

Operaciones terminales

count | sum
average
findAny | findFirst
collect
anyMatch | allMatch | noneMatch
min | max

Ejemplos practicos

Vamos a ver en detalle algunas de las operaciones de Streams siguiendo un ejemplo. Se define la clase alumno donde cada alumno conocerá los exámenes que ha rendido.



filter

Permite filtrar los elementos de un stream según el predicado que recibe como parámetro. Por ejemplo, queremos la lista de los alumnos que ingresaron en un año en particular.

```
List<Alumno> ingresantesEnAnio2020 = alumnos.stream()
    .filter(alumno -> alumno.getAnioIngreso() == 2020)
    .collect(Collectors.toList());
```

map | mapToDouble | mapToInt

Genera un stream, de igual longitud que el original, reemplazando cada elemento con el resultado de aplicar la función que recibe como parámetro sobre cada elemento del stream original. Por ejemplo, se quiere obtener la lista de los nombres de los alumnos.

```
List<String> nombresDeAlumnos = alumnos.stream()
    .map(alumno -> alumno.getNombre())
    .collect(Collectors.toList());
```

Cuando trabajamos con tipos primitivos debemos considerarlo de la siguiente manera:

```
exámenes.stream()
    .mapToInt(examen -> examen.getNota())
```

```
exámenes.stream()
    .mapToDouble(alumno -> alumno.getPromedio())
```

Ejemplo: se quiere una cantidad fija de los alumnos

```
List<Alumno> primeros3alumnos = alumnos.stream()
    .limit(cantidad)
    .collect(Collectors.toList());
```

sorted

Genera un stream con los elementos ordenados.

Ejemplo: se quieren ordenar los alumnos por promedio en forma descendente.

```
List<Alumno> alumnosOrdenadosPorPromedioDesc = alumnos.stream()
    .sorted((a1, a2) -> Double.compare(a2.getPromedio(), a1.getPromedio()))
    .collect(Collectors.toList());
```

count

Retorna la cantidad de elementos en el stream. El tipo de retorno es long. Debe hacerse la conversión a int de ser necesario.

Ejemplo: se quiere obtener la cantidad de alumnos con un promedio mayor a cierta nota.

```
int cantidadDeAlumnosConPromediosMayorA7 = (int) alumnos.stream()
    .filter(alumno-> alumno.getPromedio() >= 7 )
    .count();
```

sum

Retorna un numero que es la suma de los elementos del stream. Este debe contener números (DoubleStream, IntStream...).

Ejemplo: se quiere retornar la cantidad de exámenes tomados en un año dado.

```
int totalExamenosTomadosEn2020 = alumnos.stream()
    .mapToInt(alumno -> alumno.cantidadExamenosRendidos(2020))
    .sum();
}
```

average

Retorna un Optional con el promedio de los elementos de un stream. El objeto Optional no tiene valor si el Stream esta vacío. El stream debe ser de números (DoubleStream, IntStream...).

Ejemplo: se quiere obtener el promedio de un alumno.

```
double getPromedio = examenos.stream()
    .mapToDouble(e -> e.getNota())
    .average().orElse(0);
}
```

max | min

Retorna un Optional con el elemento máximo del stream de acuerdo al comparador indicado como parámetro. Retornara null si no hay elementos en la variable alumnos.

```
Alumno mejorPromedio = alumnos.stream()
    .max({a1, a2}->Double.compare(a1.getPromedio(), a2.getPromedio())) .orElse(null);
```

collect

Lo utilizaremos para convertir los elementos de un stream en una lista. Ya vimos anteriormente varios ejemplos en los casos de las operaciones intermedias.